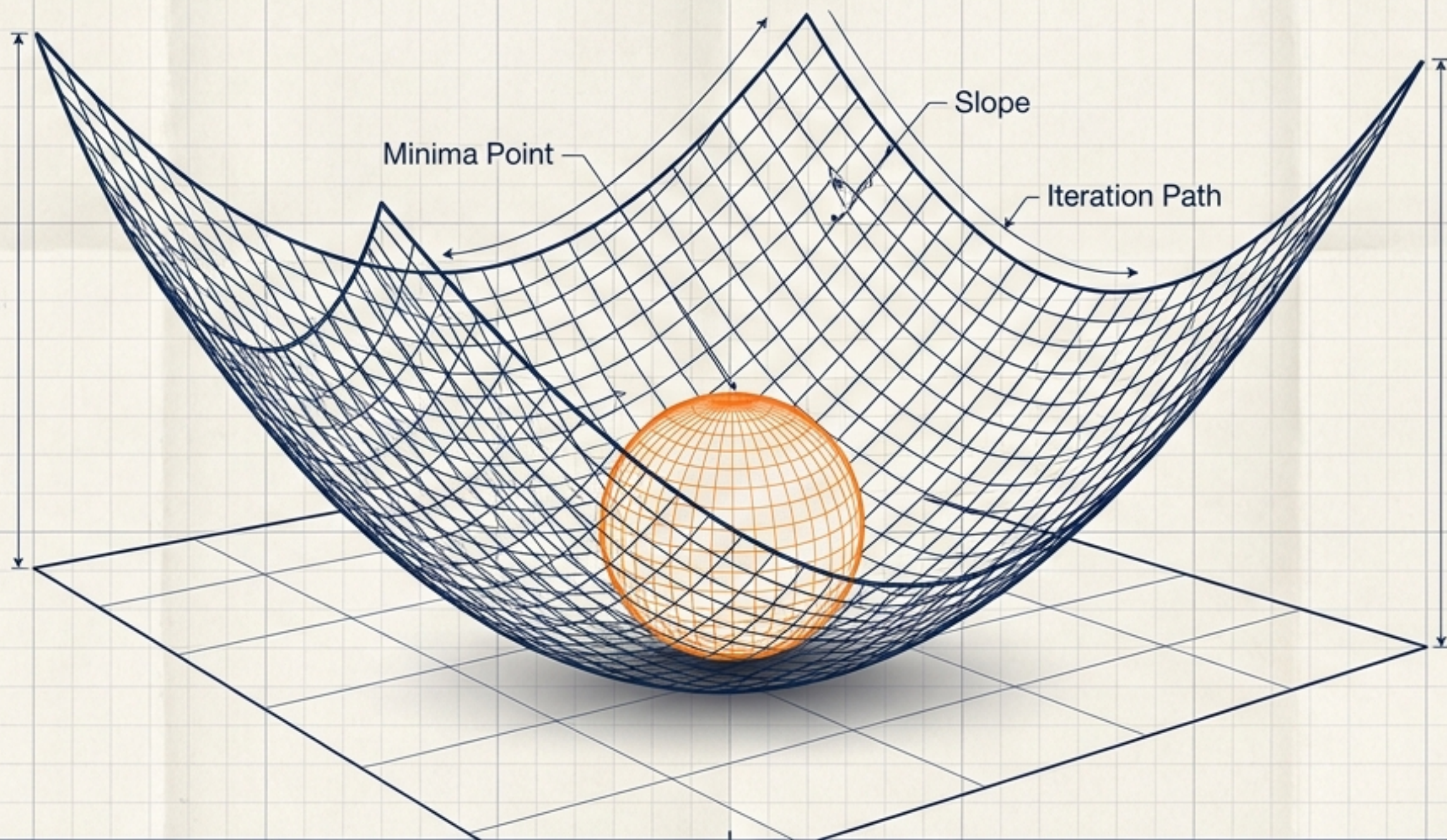


Gradient Descent

The Optimization Engine of Machine Learning



Project

Title

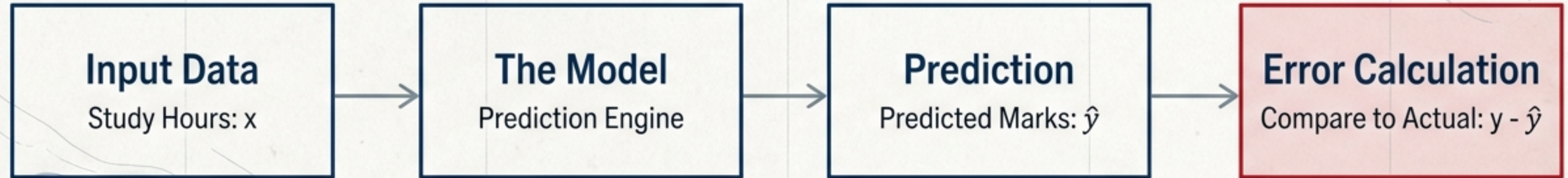
Title

Date

Time

The Machine Learning Goal

Our objective is not just to generate predictions. The goal is to optimize the model by finding the exact mathematical point where its overall error is at its absolute minimum.



The Shape of Our Mistakes

Because we square our errors in the MSE formula, the Cost Function forms a non-linear U-shaped curve. Squaring turns all negative errors positive and heavily penalizes large mistakes. Our mission is to navigate to the bottom of this shape.



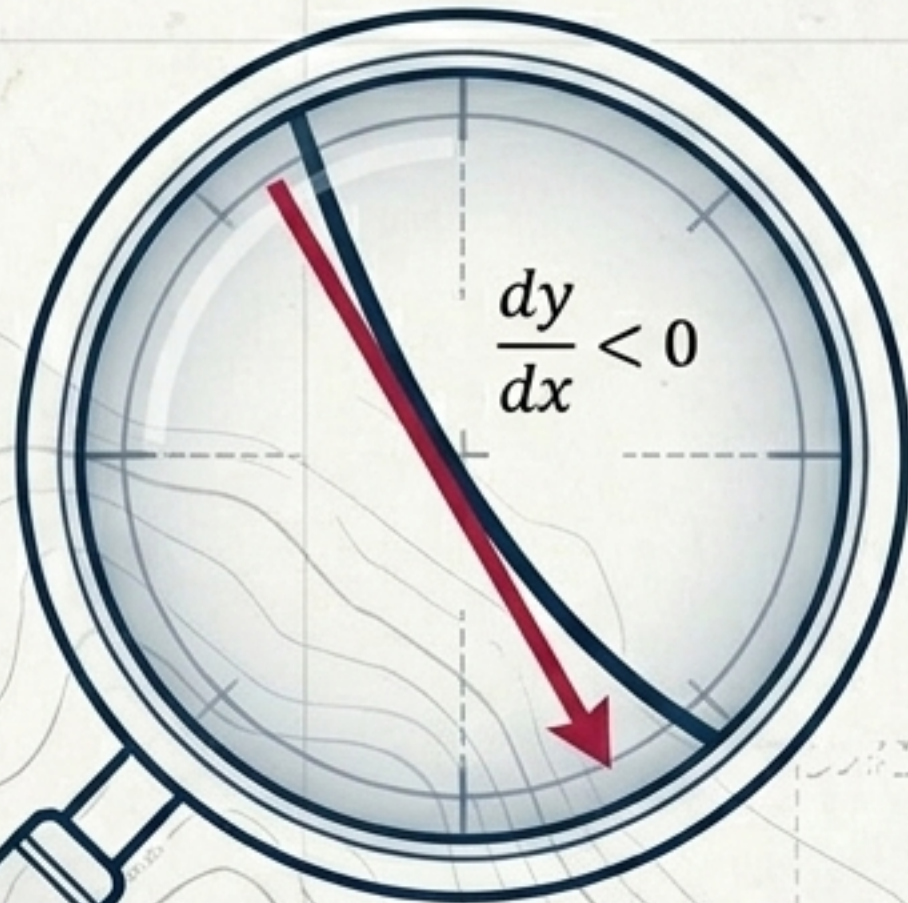
Mean Squared Error (MSE):

$$J = \frac{1}{M} \sum (Actual - Prediction)^2$$

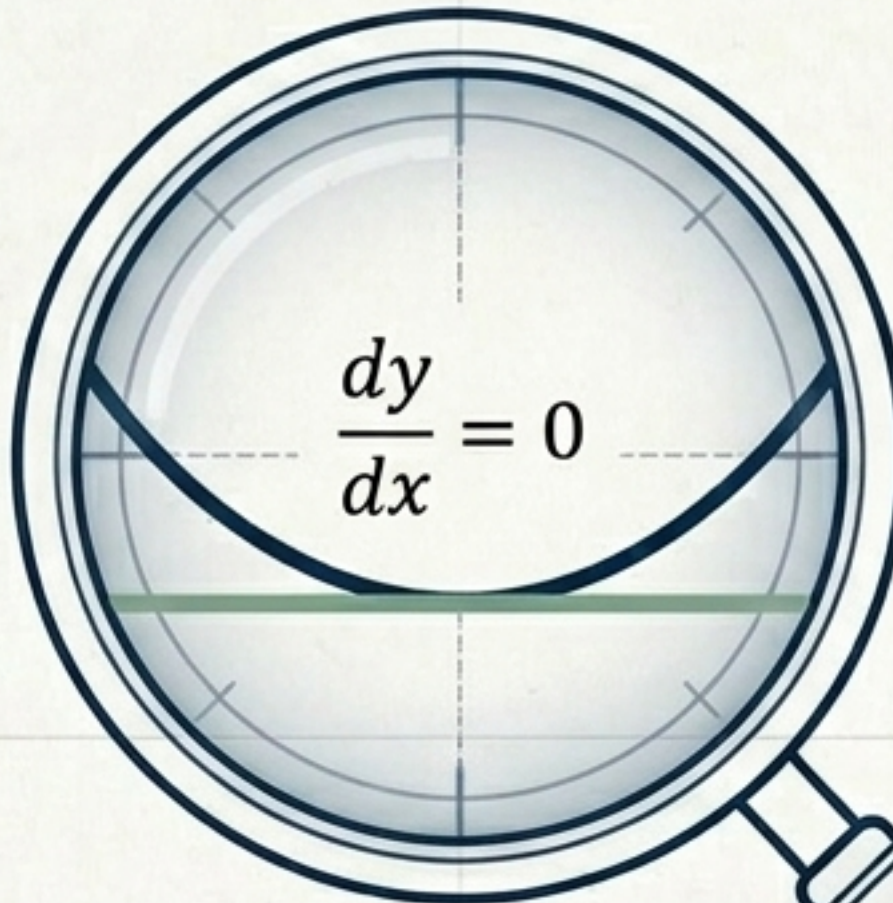
How Do We Find the Bottom?

We use derivatives (Rate of Change) as a slope detector. The lowest point of the curve is the exact moment the tangent line becomes perfectly flat.

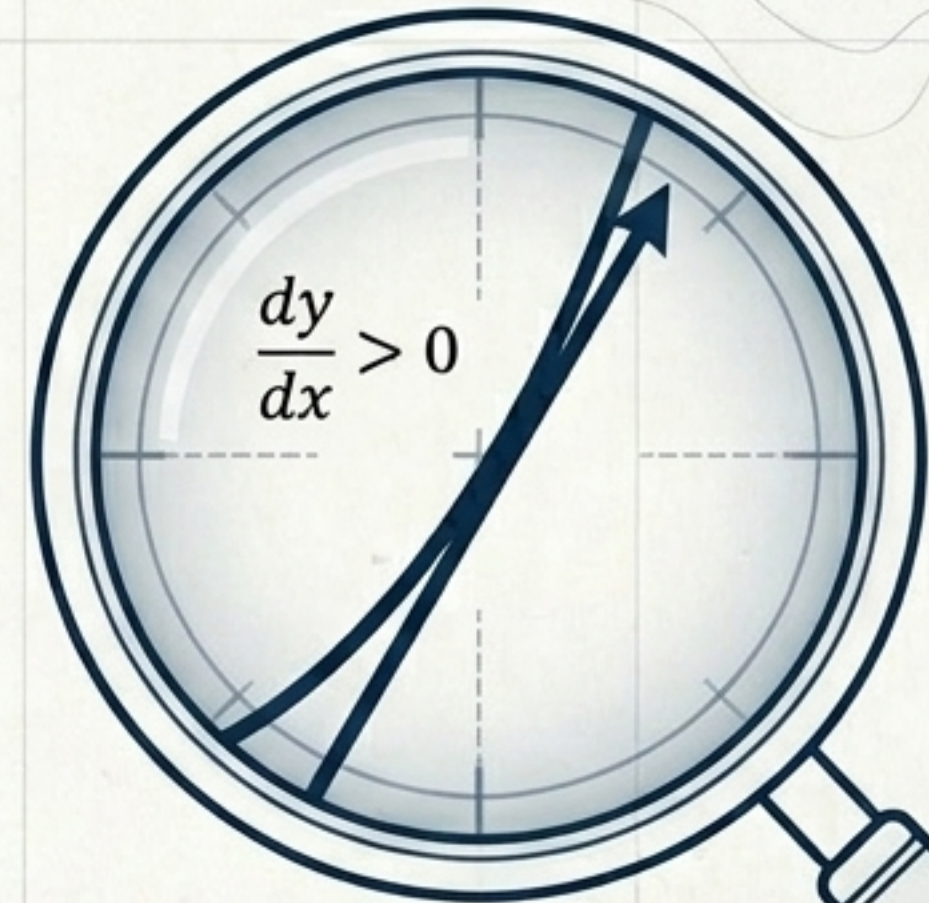
High Negative Slope



Slope = $\frac{dy}{dx} = 0$



High Positive Slope



Turning Math into Location

1. The Machine (Derivative)

$$y = x^2$$

$$\frac{dy}{dx} = 2x$$

The derivative provides a formula to measure the slope at any point.

2. The Location (X-axis)



$$2x = 0 \Rightarrow x = 0$$



$$x = 0$$



$$y = 0^2 \Rightarrow y = 0$$



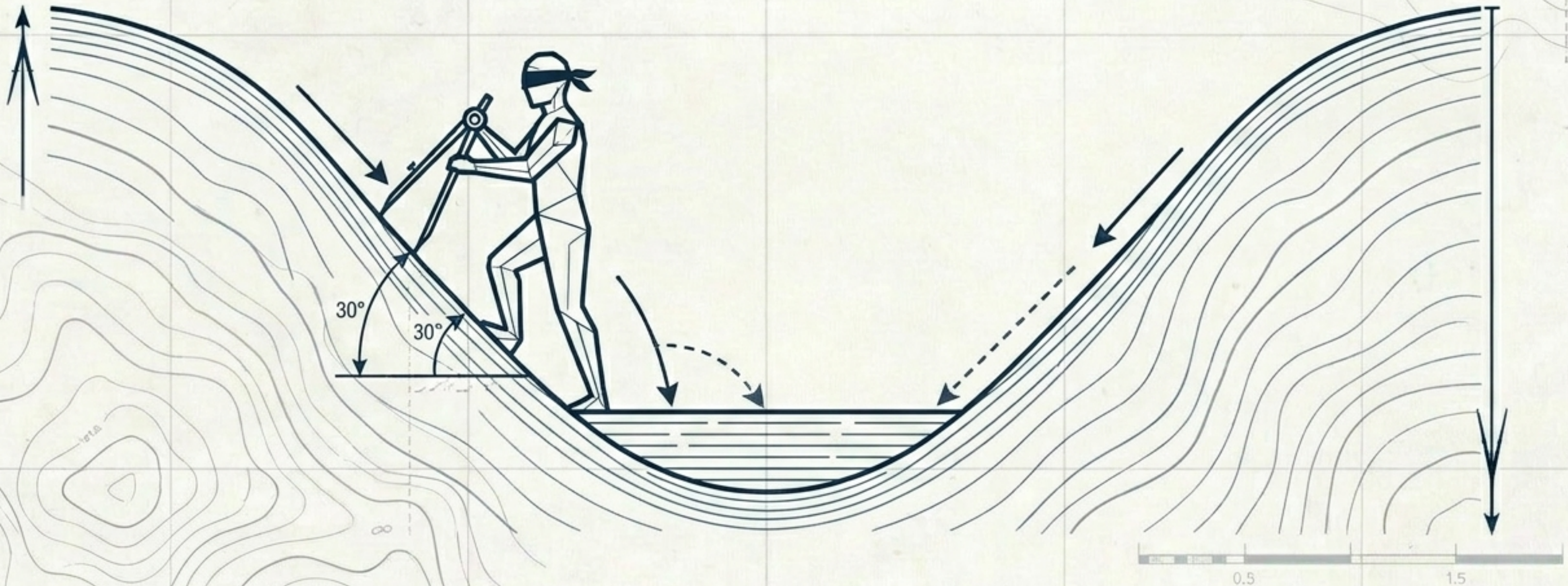
$$y = 0$$

Setting the derivative to 0 reveals the exact X-axis coordinate of the valley floor.

Plugging the location back into the original formula gives us the absolute minimum Error.

Walking Down the Hill Blindfolded

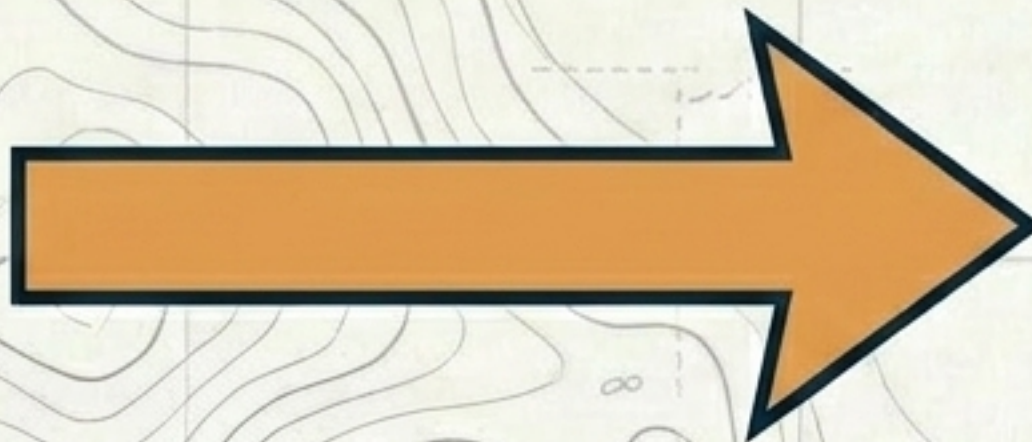
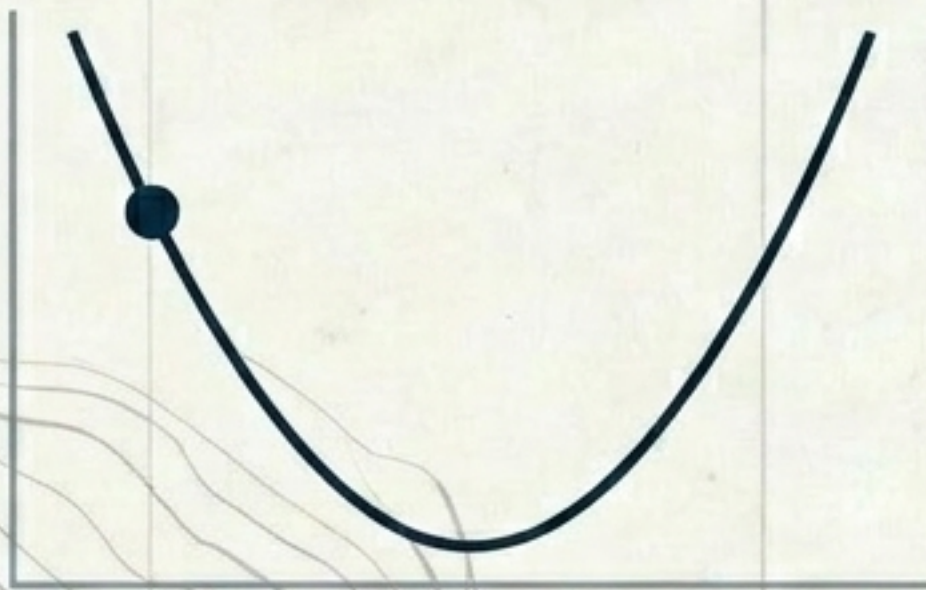
Gradient Descent starts by dropping the model at a random coordinate. Unable to see the bottom, the algorithm checks the slope under its feet, takes a step in the opposite direction of the incline, and repeats this process until the ground is perfectly flat.



Which Way Do We Step?

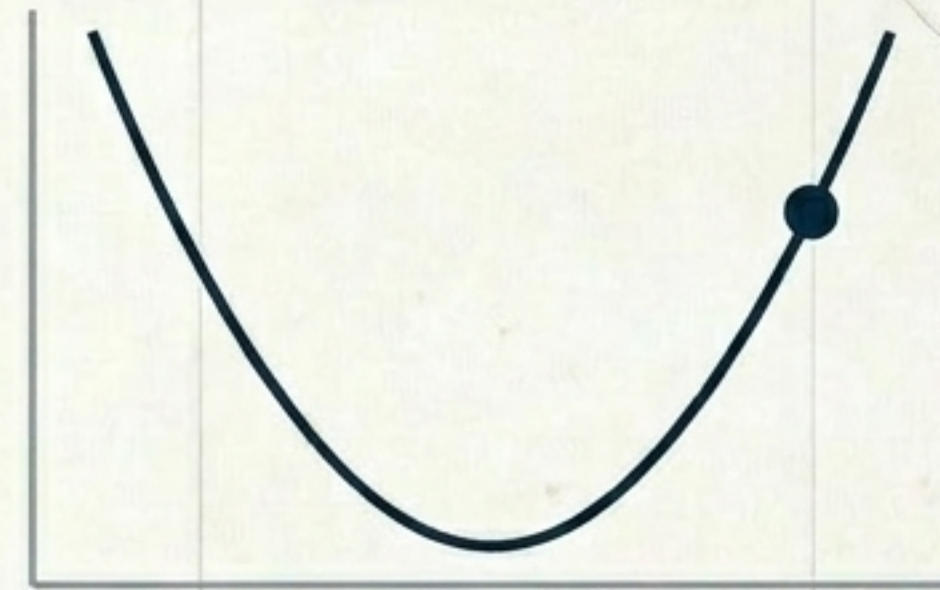
Case A: Negative Slope

$x \uparrow \Rightarrow y \downarrow$
Increase x to descend



Case B: Positive Slope

$x \uparrow \Rightarrow y \uparrow$
Decrease x to descend



The Engine of Optimization

The Assignment

Update our current position.

The diagram shows the equation $x := x - dy/dx$ in a large, stylized font. Four callout lines with dots at the end point to specific parts of the equation: one points to the first x (labeled 'The Assignment'), one points to the second x (labeled 'The Anchor'), one points to the minus sign (labeled 'The Gradient'), and one points to the dy/dx term (labeled 'The Inverter').

$$x := x - dy/dx$$

The Anchor

Where we currently stand.

The Gradient

The slope of the ground beneath us.

The Inverter

The built-in direction corrector.

Built-In Navigation: The Magic of the Minus Sign

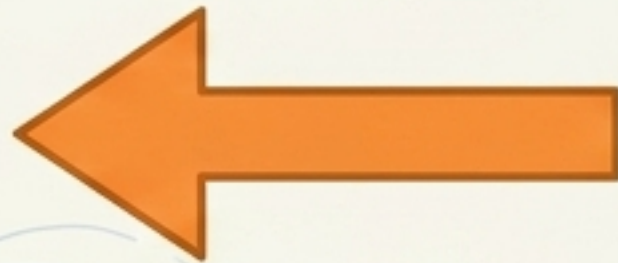
The formula **auto-corrects**. A **negative slope** creates a double-negative, pushing us right. A positive slope triggers subtraction, pulling us left.

Starting on the Right (Positive Slope)

$$\text{Start: } x = 3$$

$$\text{Derivative (2x): } 2(3) = +6$$

$$\text{Update: } x_{\text{new}} = 3 - (+6) = -3$$



Result: Moves Left (Decreases x)

Starting on the Left (Negative Slope)

$$\text{Start: } x = -2$$

$$\text{Derivative (2x): } 2(-2) = -4$$

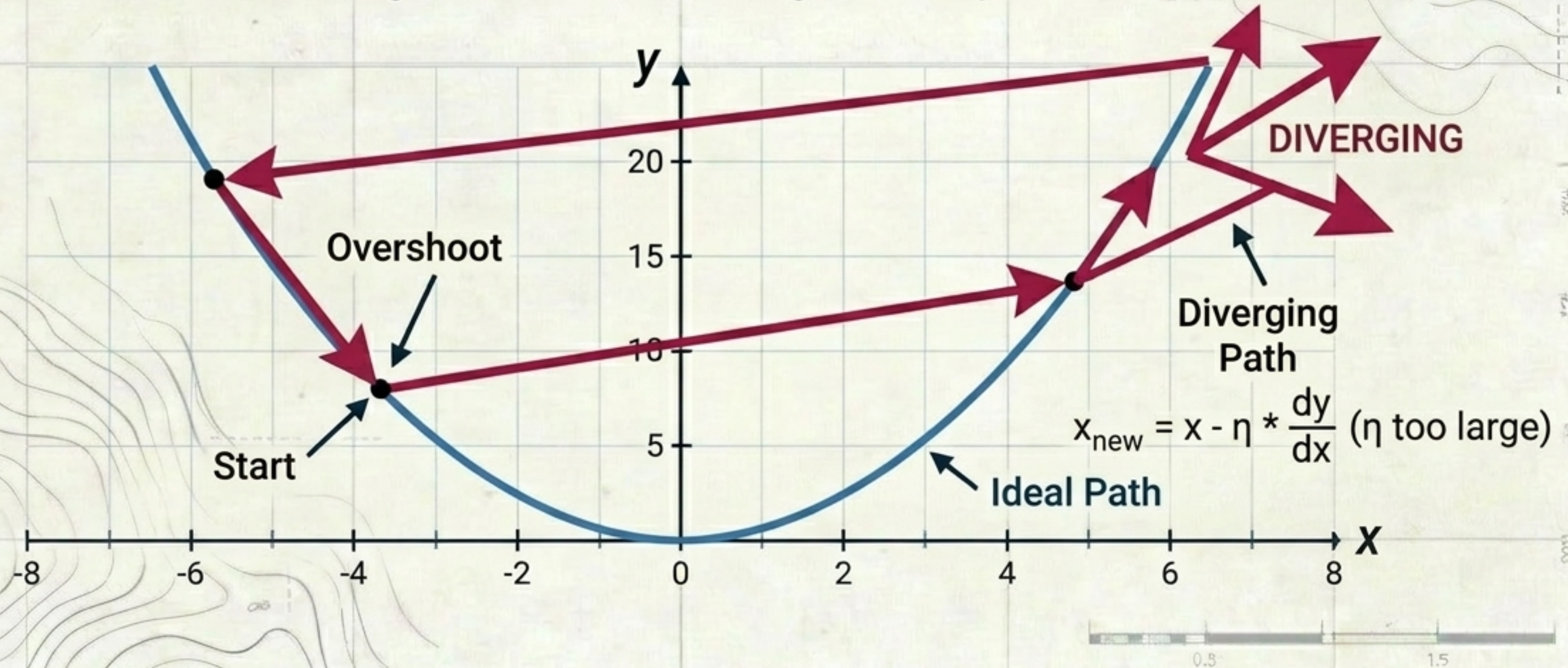
$$\text{Update: } x_{\text{new}} = -2 - (-4) = -2 + 4 = +2$$



Result: Moves Right (Increases x)

The Danger of Taking Giant Leaps

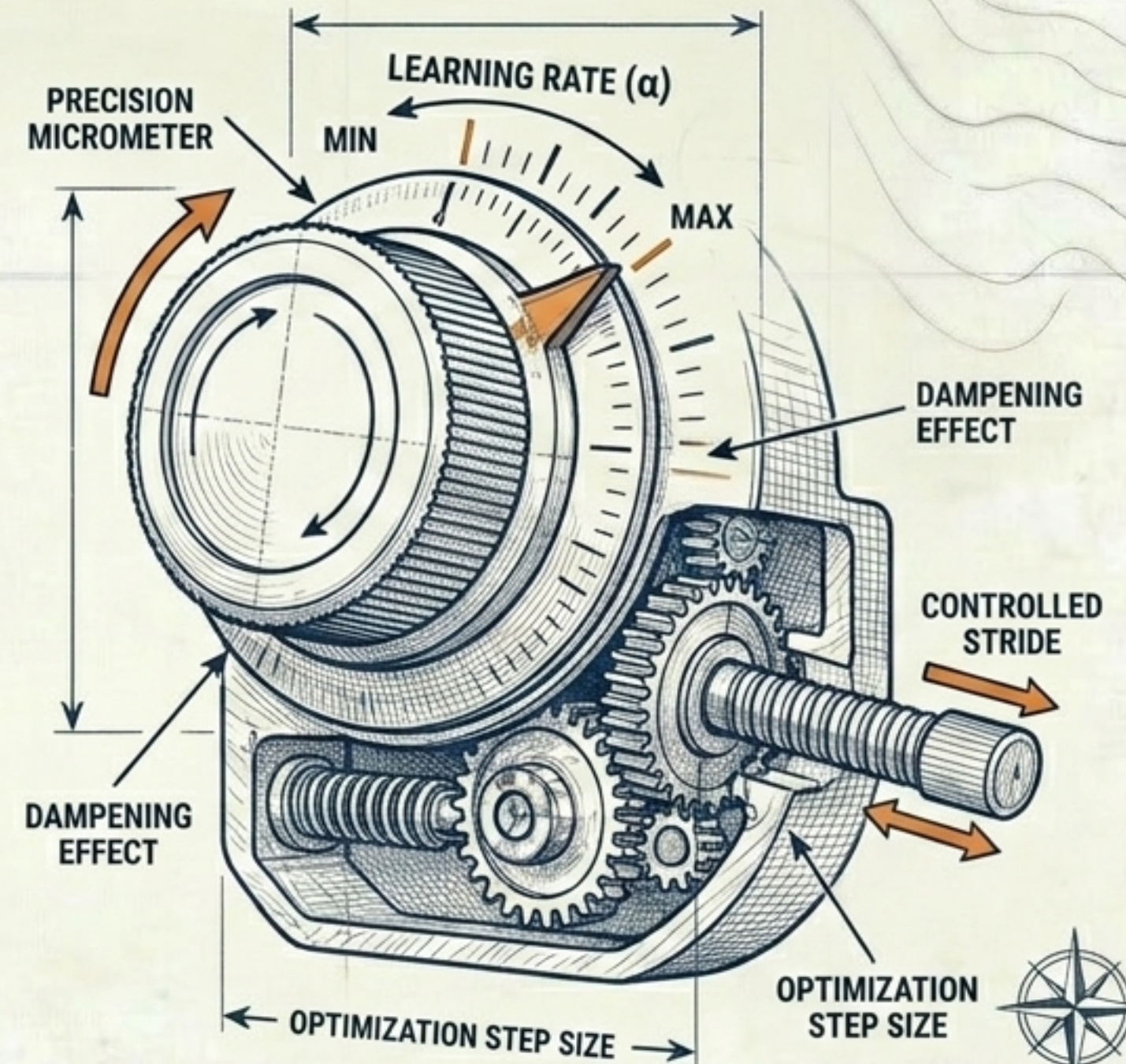
If the slope is incredibly steep, the basic update equation takes a step so massive that it completely bypasses the minimum point. The model begins bouncing back and forth, failing to ever optimize.



The Control Dial: Learning Rate (α)

$$\mathbf{x} := \mathbf{x} - \alpha \left(\frac{dy}{dx} \right)$$

Alpha acts as a dampener. It scales the size of the step, ensuring that regardless of how extreme the gradient is, the algorithm takes measured, controlled strides toward the minimum.



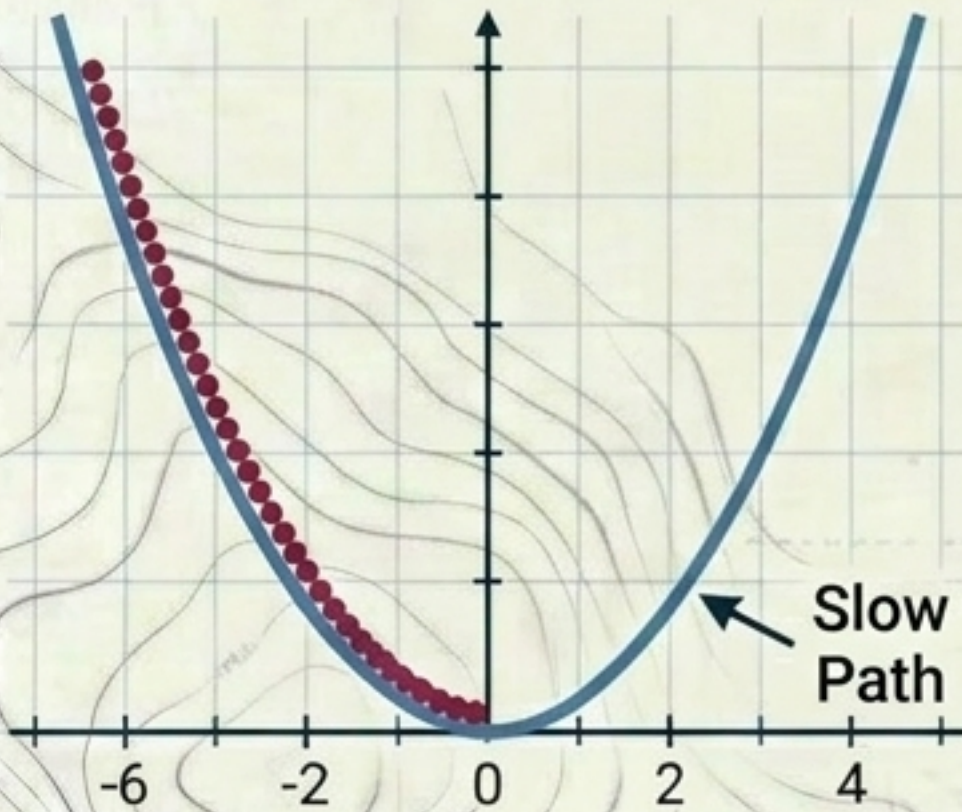


Tuning Alpha: The Goldilocks Problem

α Too Small

Step size: $0.00001 \times 5 = 0.00005$

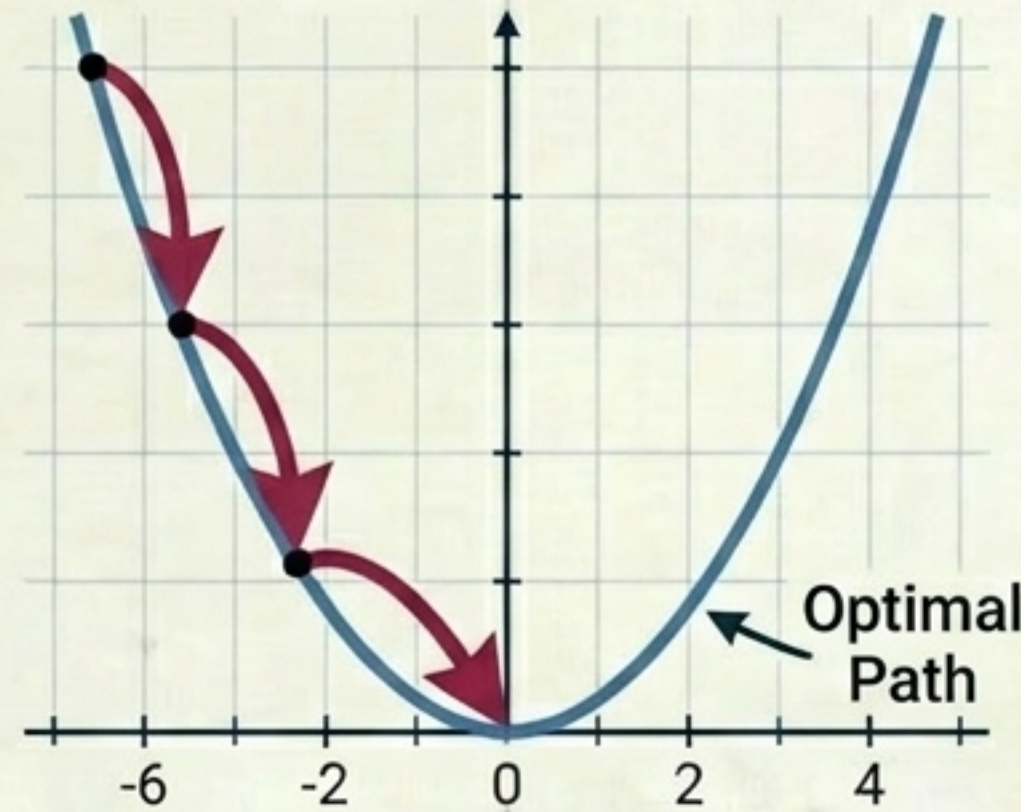
Diagnosis: Slow Convergence



α Just Right

Step size: Moderate

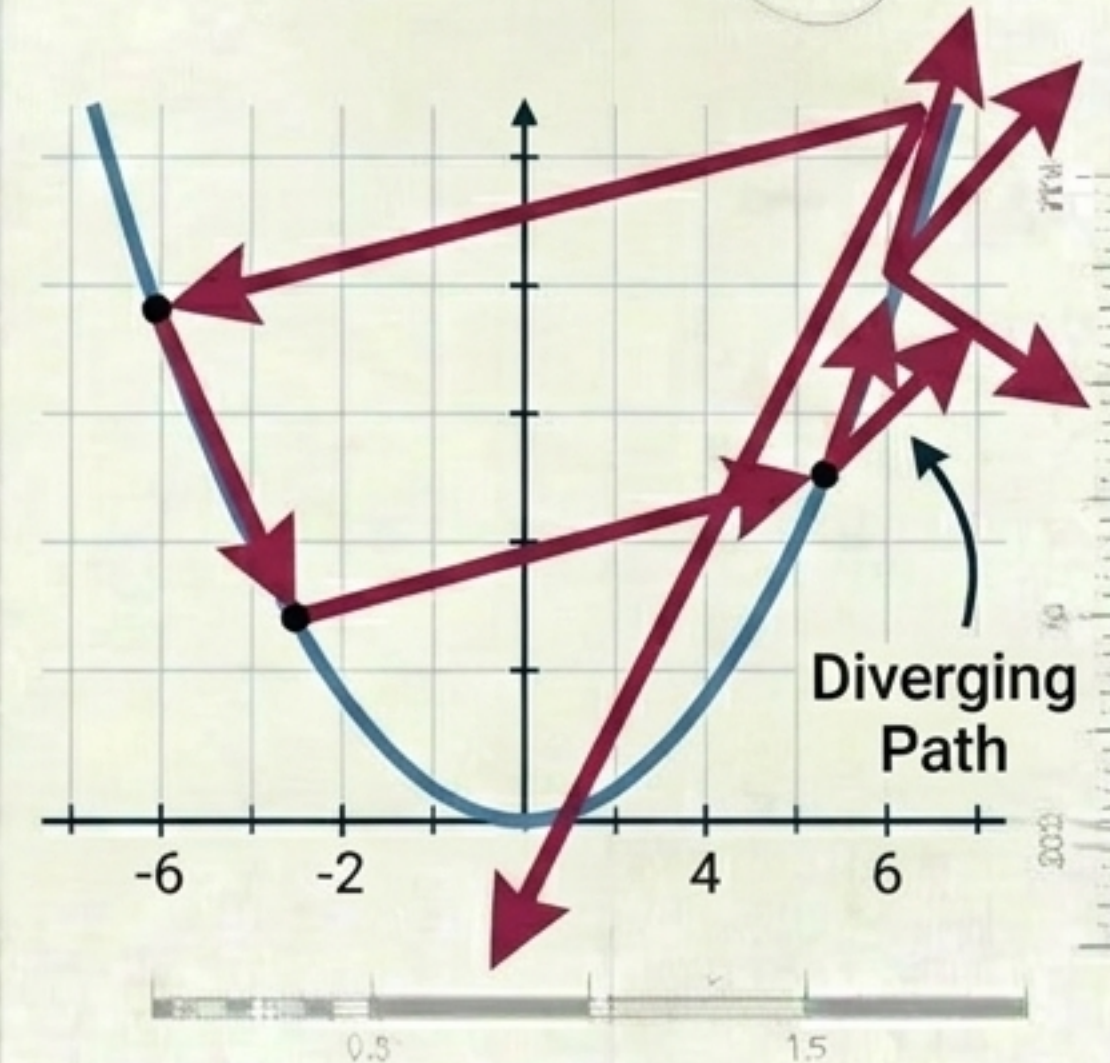
Diagnosis: Optimal Convergence



α Too Large

Step size: $2 \times 5 = 10$

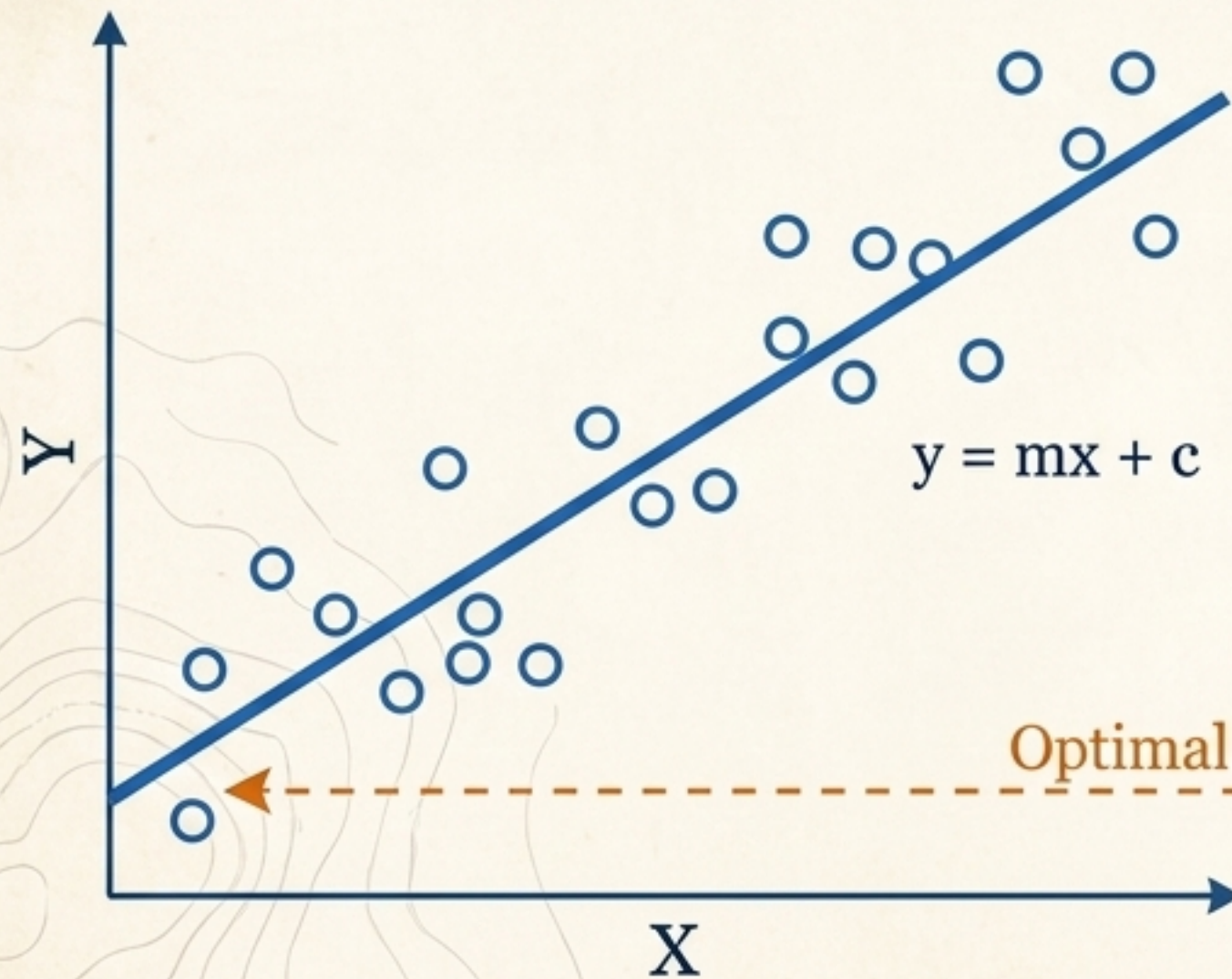
Diagnosis: Divergence



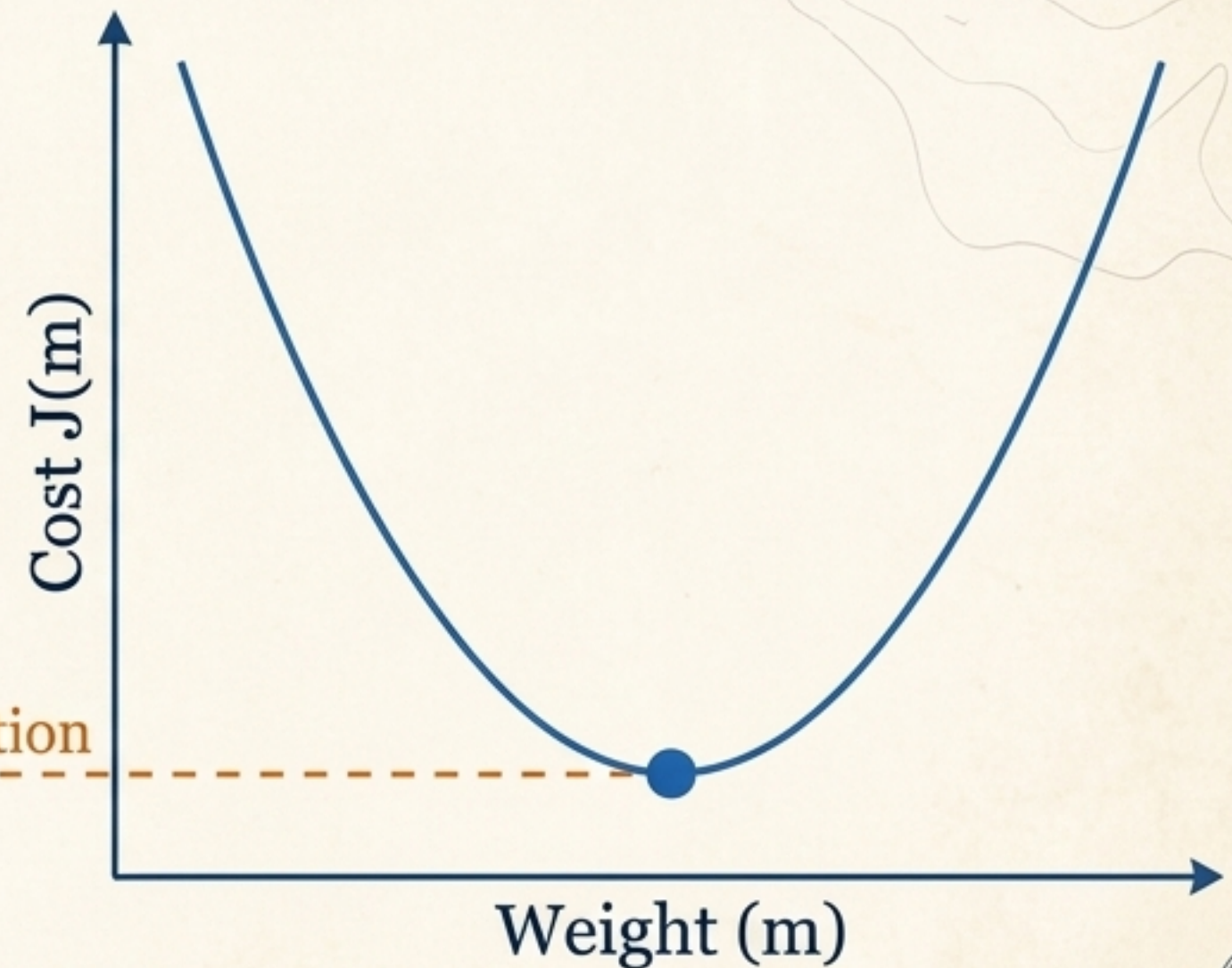
Finding the Perfect Line

In Linear Regression, Gradient Descent continuously updates the weight (m). As m descends to the bottom of the Cost Function curve, our regression line automatically locks into the perfect fit for the data.

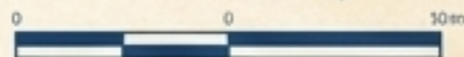
A. Linear Regression Scatterplot



B. Cost Function (J)



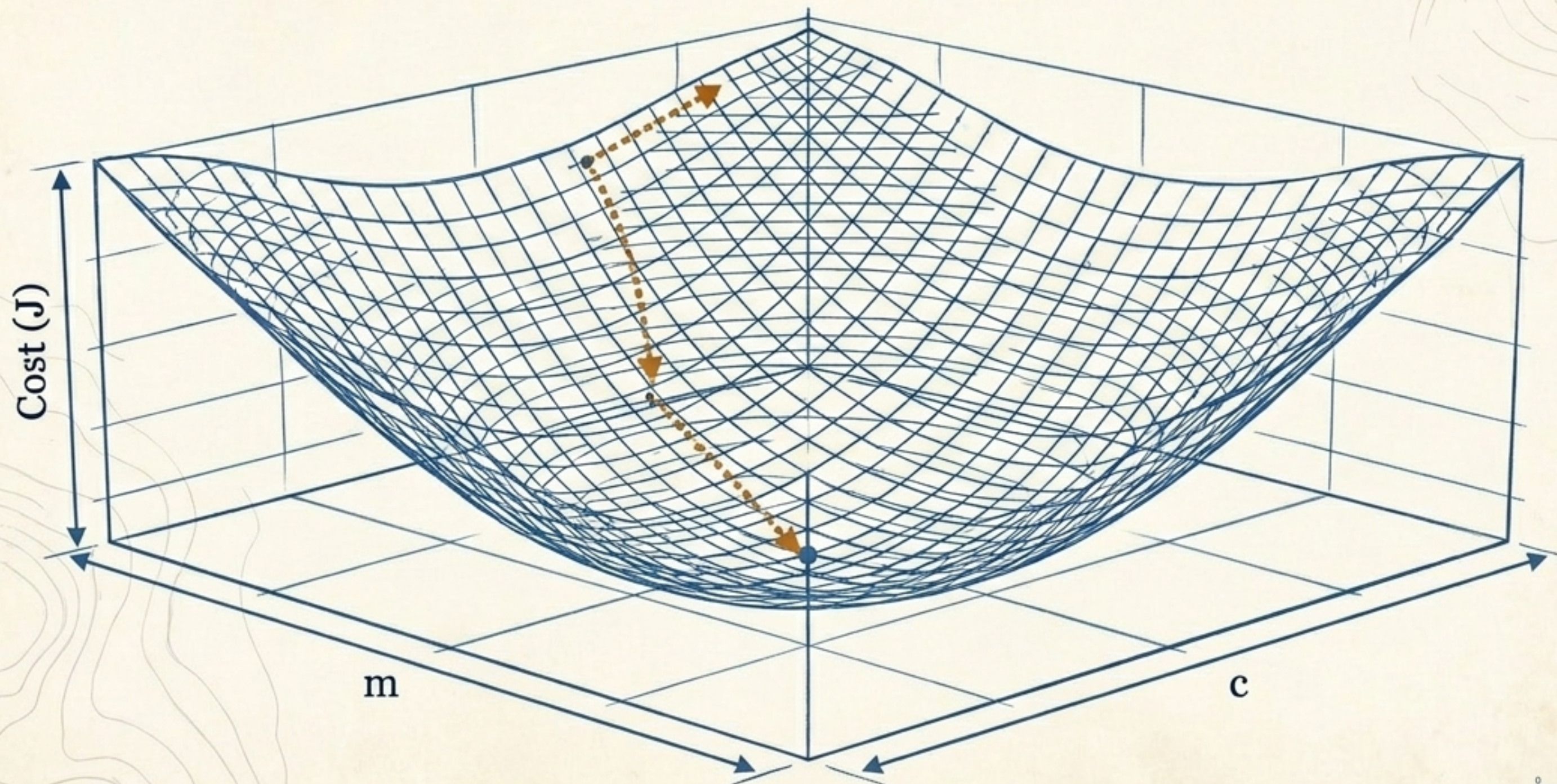
$$\text{Update Rule: } m := m - \alpha * d(J)/dm$$





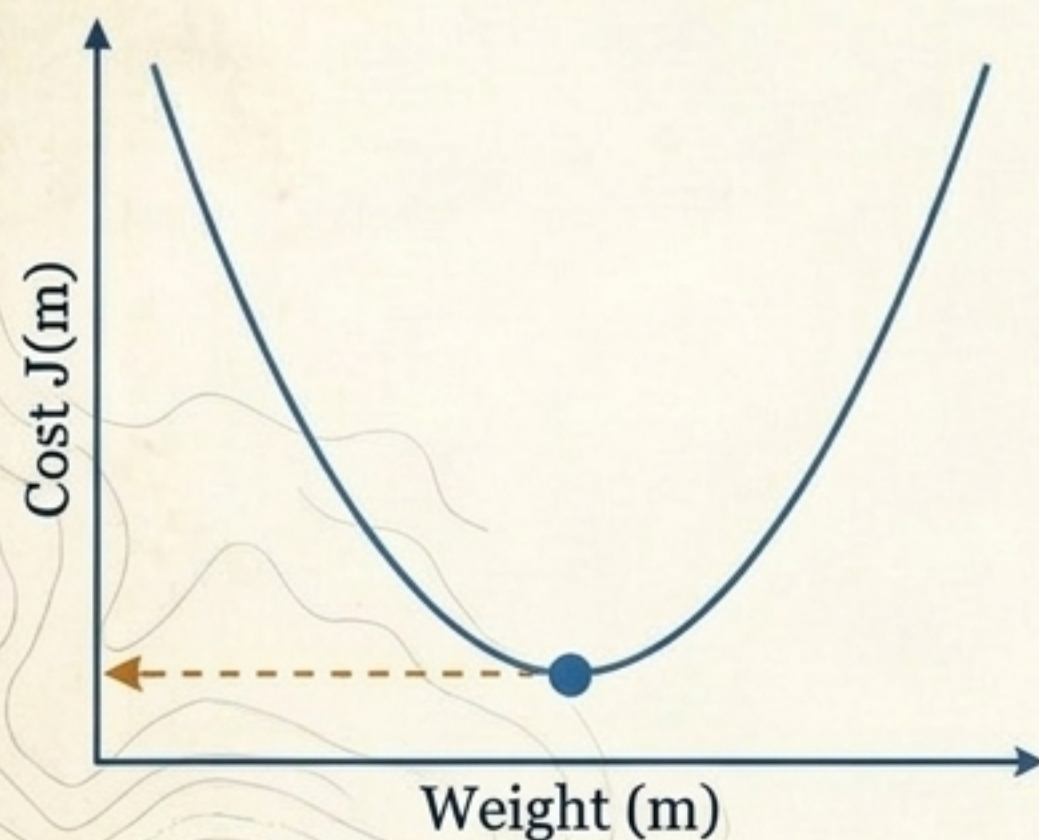
Navigating in 3D Space

Real models have multiple variables. The algorithm evaluates the North/South slope and the East/West slope simultaneously. It then takes a diagonal step directly toward the lowest point of the 3D bowl.

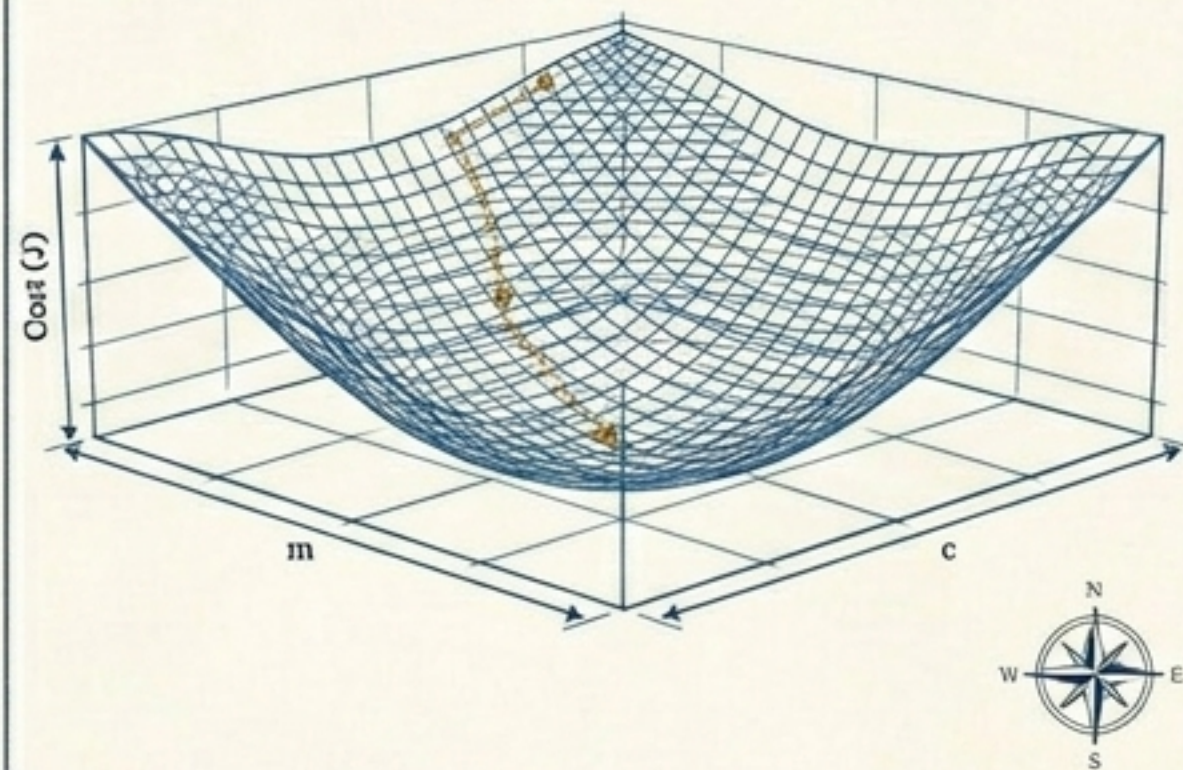


One Logic, Infinite Dimensions

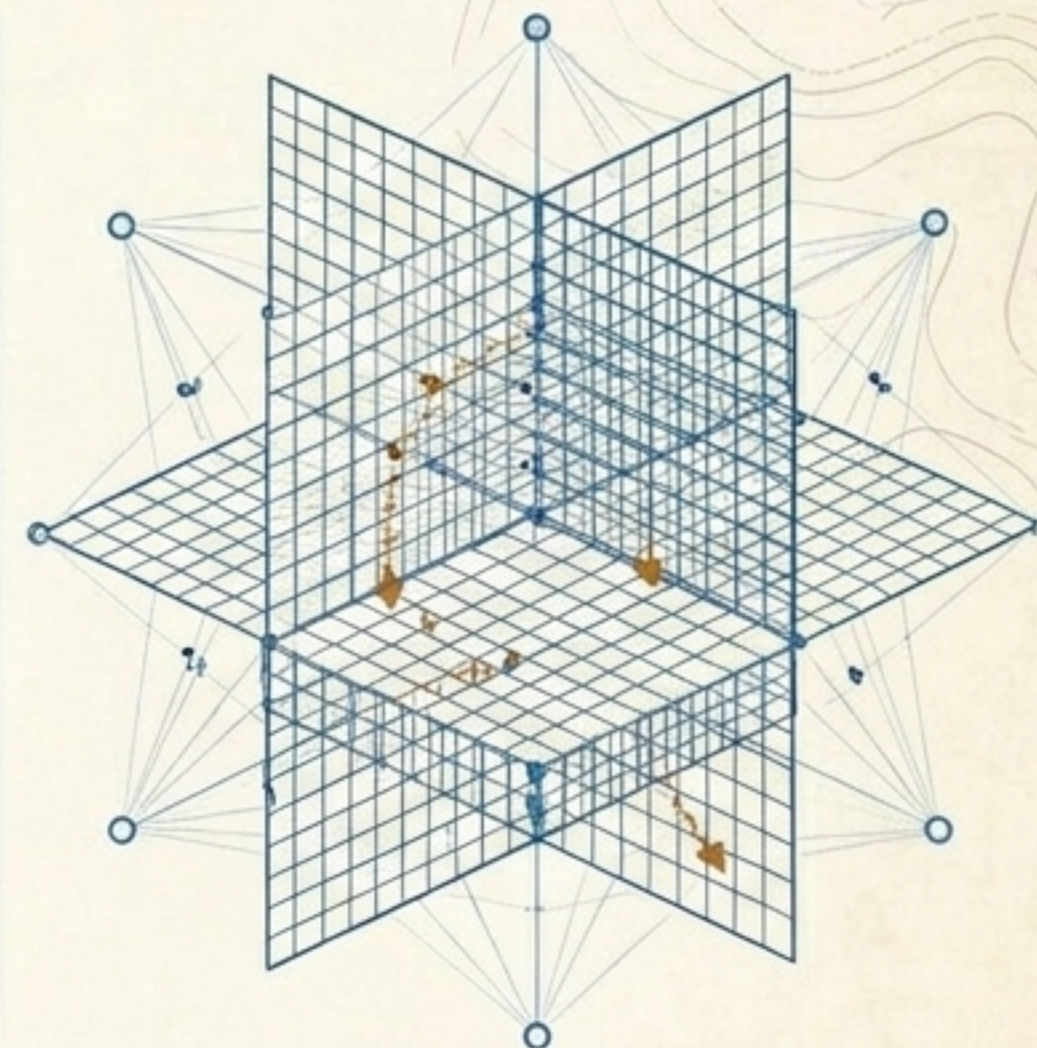
Whether updating a single weight on a curve, two weights in a 3D bowl, or billions of parameters in a neural network, the engine remains exactly the same. Calculate the gradients, update simultaneously, and descend.



1 Variable: Line



2 Variables: Plane



N-Variables: Hyperplane

Calculate the gradients, update simultaneously, and descend.

